

Relacionamentos entre Objetos & POO no Back-End

GRUPO 4

Jacó de Souza Rufino

Gabriel Lúcio de Araújo

O que Vamos Explicar Hoje?

As conexões entre objetos:

Associação, Agregação e Composição.



A diferença crucial:

Ciclos de vida e dependência de dados.

Modelagem no Back-End:

Entidades de Domínio e Classes de Serviço.

Demonstração prática de um e-commerce em JavaScript.

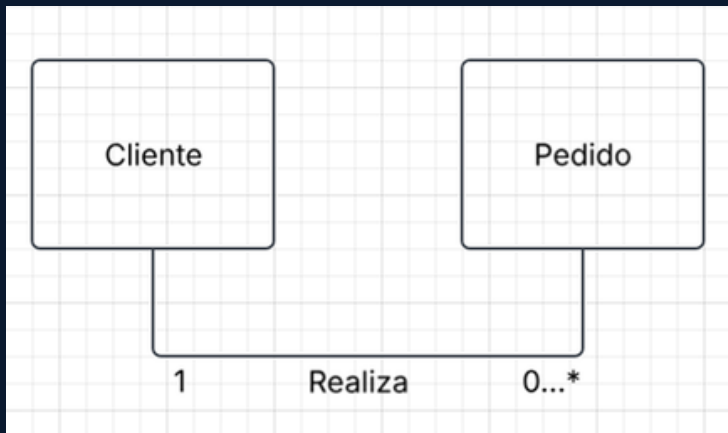
Associação, Agregação e Composição

ASSOCIAÇÃO

Conexão simples e livre. Os objetos são completamente independentes.

EXEMPLO

Cliente ↔ **Pedido**

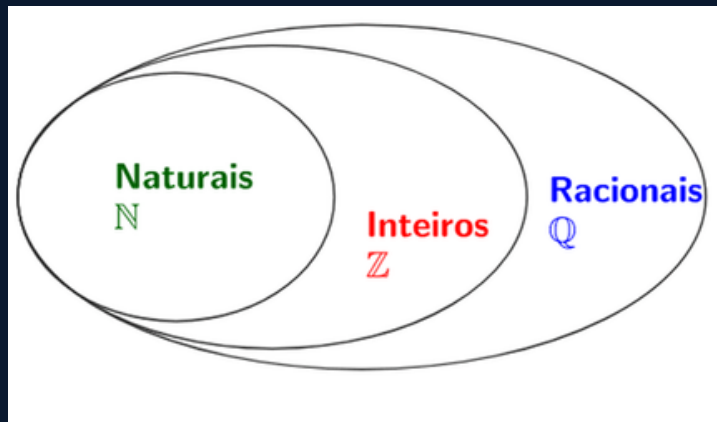


AGREGAÇÃO

Relação "todo-parte" onde a parte pode existir separadamente (relação fraca).

EXEMPLO

Time → **Jogadores**



COMPOSIÇÃO

Relação "todo-parte" onde a parte pertence rigidamente ao todo e não tem vida independente (relação forte).

EXEMPLO

Pedido → **Itens do Pedido**



Organização do Servidor: Entidades e Serviços

Entidades do Domínio: Classes que modelam conceitos reais do negócio e guardam os dados e estados do sistema (ex: Usuario, Produto, Pedido).

Classes de Serviço (Services): Classes que não representam dados, mas concentram comportamentos, lógica e regras de negócio complexas (ex: PedidoService, UsuarioService).

A Conexão: Modelando Bancos de Dados e Regras

Associações:

Geram chaves estrangeiras
(cliente Id conecta um
pedido a um cliente).

Composições:

Definem exclusões em cascata
(se um Pedido for excluído, os
itens vinculados a ele devem ser
excluídos automaticamente).

As Classes de Serviço manipulam essas regras para processar requisições com segurança.

Analogia dos Relacionamentos

Associação

Gera **Chaves Estrangeiras (FK)**. O `clienteId` liga o pedido ao cliente sem misturar os seus ciclos de vida.

Agregação

Entidades independentes. Jogadores podem ficar "sem time" ou ser transferidos sem serem eliminados.

Composição

Define **Exclusão em Cascata**. Se o Pedido for removido, os itens vinculados são eliminados automaticamente.

TABLE: Pedido

`id` (PK)

`cliente_id` (FK)

`data_criacao`

↓ Deletação em cascata

TABLE: ItemPedido

`id` (PK)

`pedido_id` (FK)

`produto_id`

`quantidade`

O Nosso Cenário de E-commerce



Implementação Prática em JavaScript

```
class ItemPedido {
  constructor(nome, preco) {
    this.nome = nome;
    this.preco = preco;
  }
}

class Pedido {
  constructor(id, clienteId) {
    this.id = id;
    this.clienteId = clienteId; // Associação
    this.itens = []; // Composição
  }

  adicionarItem(nome, preco) {
    // A criação do ItemPedido ocorre dentro do Pedido
    this.itens.push(new ItemPedido(nome, preco));
  }
}
```

COMPOSIÇÃO

ASSOCIAÇÃO

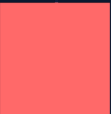
Explicação do Código e Relação com Back-End

O papel do `this`: Aponta para o objeto atual em execução, garantindo que o pedido do Cliente A não misture itens com o do Cliente B.

Onde está a Composição: No método `adicionarItem()`, onde o próprio `Pedido` cria (new) a instância de `ItemPedido`. Se o pedido for destruído, a lista `this.itens` some junto.

Onde está a Associação: No atributo `clienteId`, ligando o pedido ao cliente apenas por uma referência.

O que Devemos Lembrar?



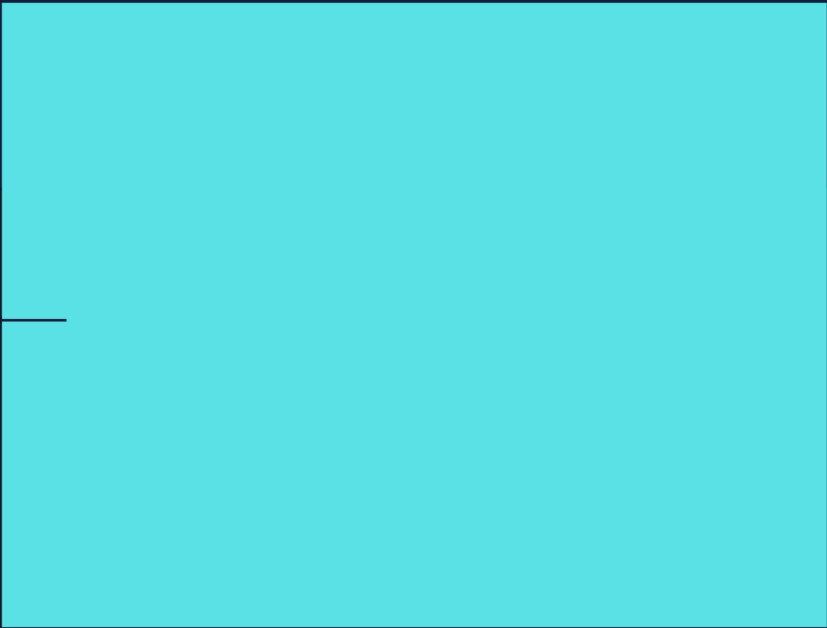
Sistemas Back-End de verdade dependem de modelagem precisa para evitar inconsistência de dados órfãos.



A divisão em Entidades (dados) e Serviços (comportamentos) mantém o servidor limpo e escalável.



Diferenciar Agregação de Composição dita as regras de exclusão no Banco de Dados.



Conclusão e Fontes

O QUE DEVEMOS LEMBRAR?

Modelagem de Relações

Associação (independente), Agregação (fraca) e Composição (forte) ditam a integridade dos dados.

Ciclo de Vida

Entender a dependência entre objetos evita a criação de dados órfãos no banco de dados.

Arquitetura Back-End

A separação entre Entidades e Serviços garante um código escalável e regras de negócio centralizadas.

Fontes Consultadas

MDN Web Docs: Classes

JavaScript Reference

MDN Web Docs: Working with Objects

JavaScript Guide

MDN Web Docs: Private Elements

JavaScript Reference

Materiais Didáticos

PDFs 01 e 02 de Apoio ao Aluno